

TCP/IP on z/OS





NETWORKS

There's no need to belabor the definition of *network*. You already know what a network is. You may know more about them than your instructor does (although that isn't a high bar).

At a certain level of abstraction, we can say a network consists of

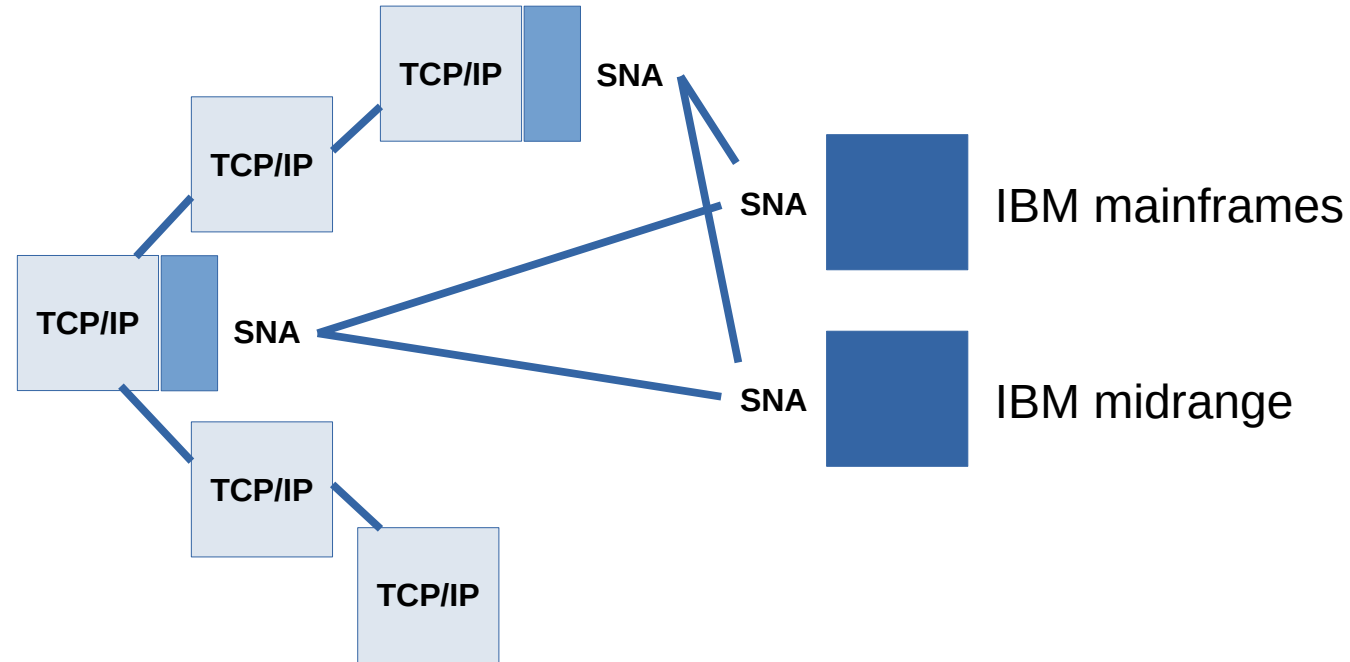
- hardware,
- software; and
- protocols



The IBM Z can be configured with the hardware components necessary for it to communicate with other network nodes. The z/OS software that supports networking includes two stacks: TCP/IP and SNA. TCP/IP is the older of the two and is the global standard. SNA (Systems Network Architecture) is IBM's legacy networking system.

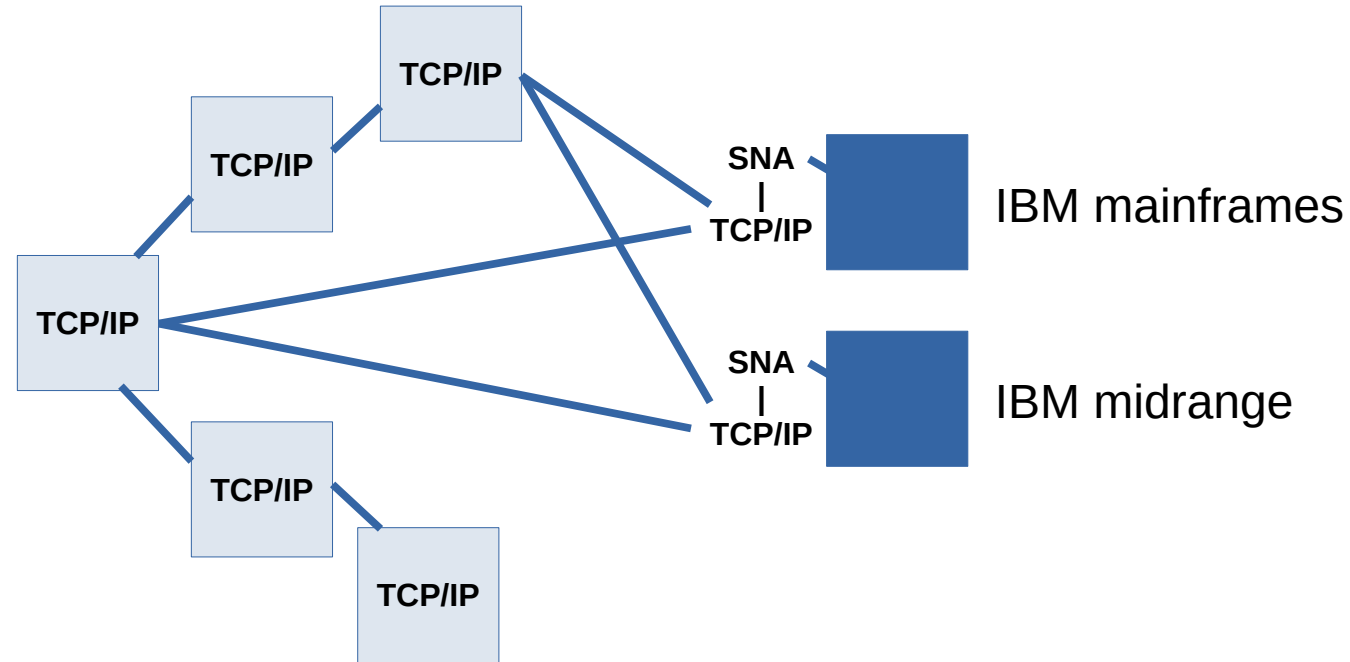
I suggest that what makes a network a network are the protocols. Those are the rules of engagement between network nodes.

FROM SNA TO TCP/IP



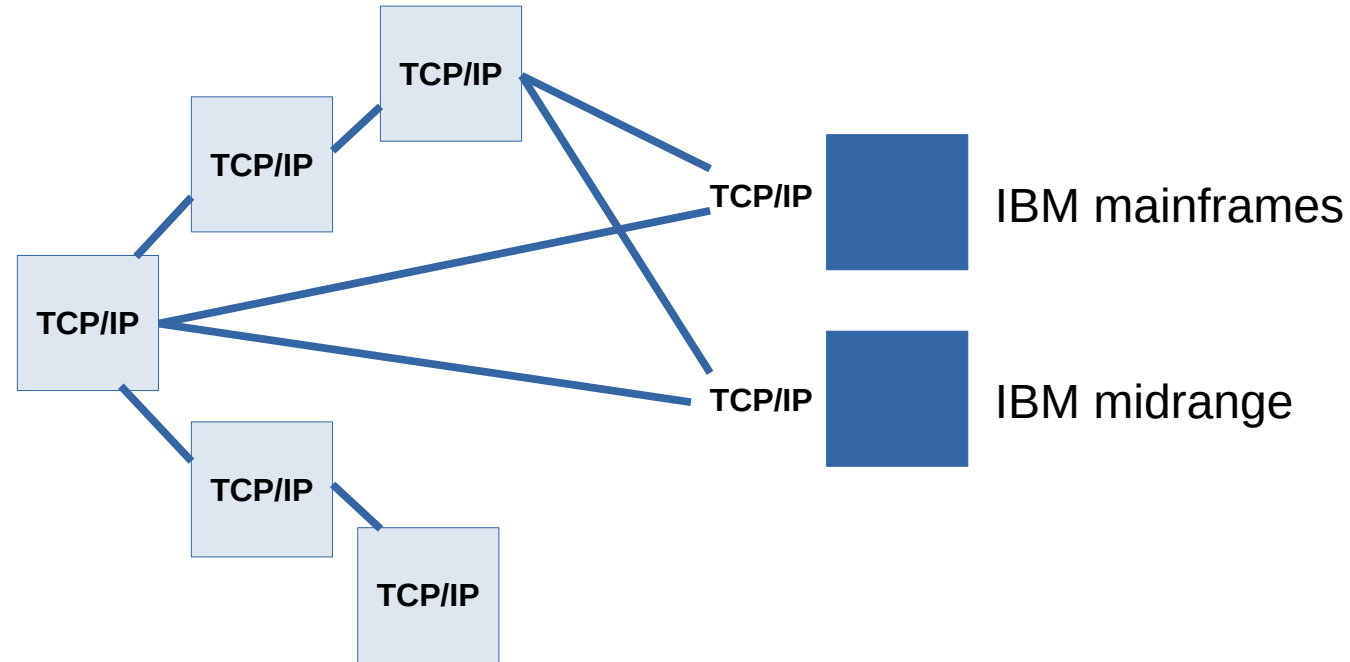
Long ago, IBM dominated the computer market. They could dictate the rules of engagement for networking. While the rest of the world was adopting TCP/IP, IBM stuck with its own networking solution, SNA (Systems Network Architecture). If you wanted to connect to IBM systems, you had to translate between TCP/IP and SNA.

FROM SNA TO TCP/IP



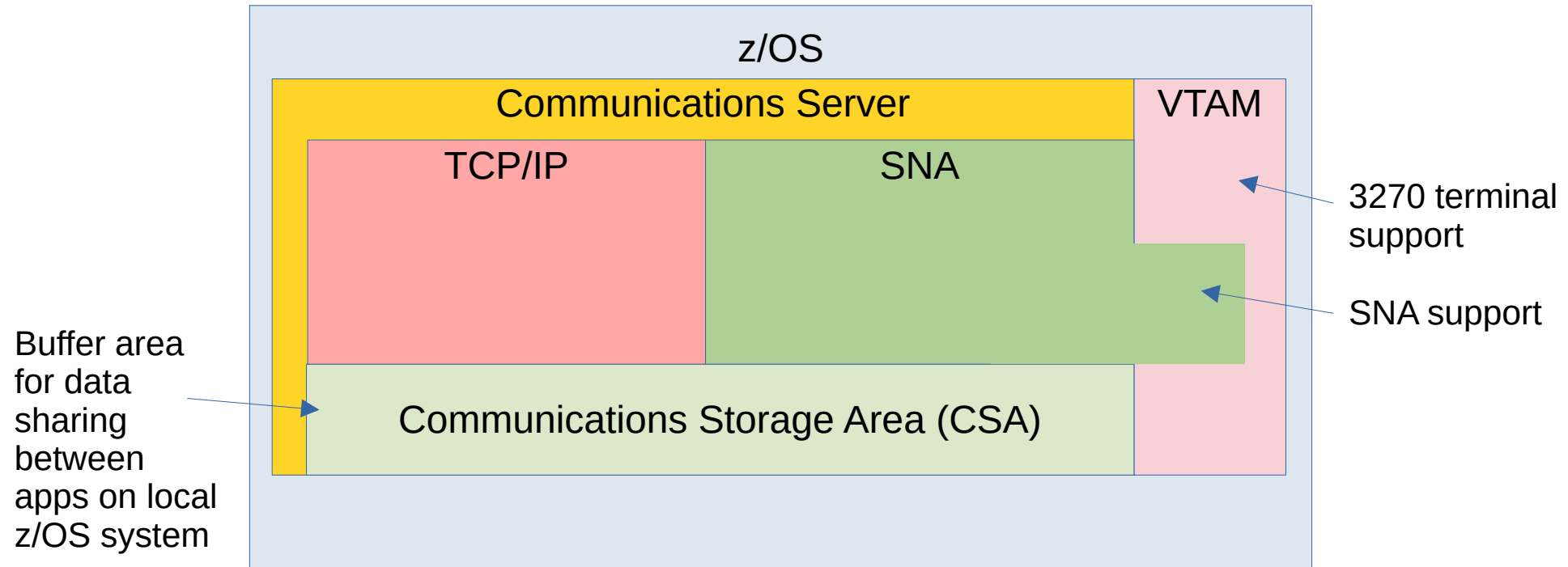
When the distributed computing model was becoming the standard, and customers were demanding simpler and more-seamless interconnectivity between disparate systems, IBM began to integrate TCP/IP into the SNA stack. Initially, they added a translation layer. Now, customers didn't have to translate to/from SNA.

FROM SNA TO TCP/IP



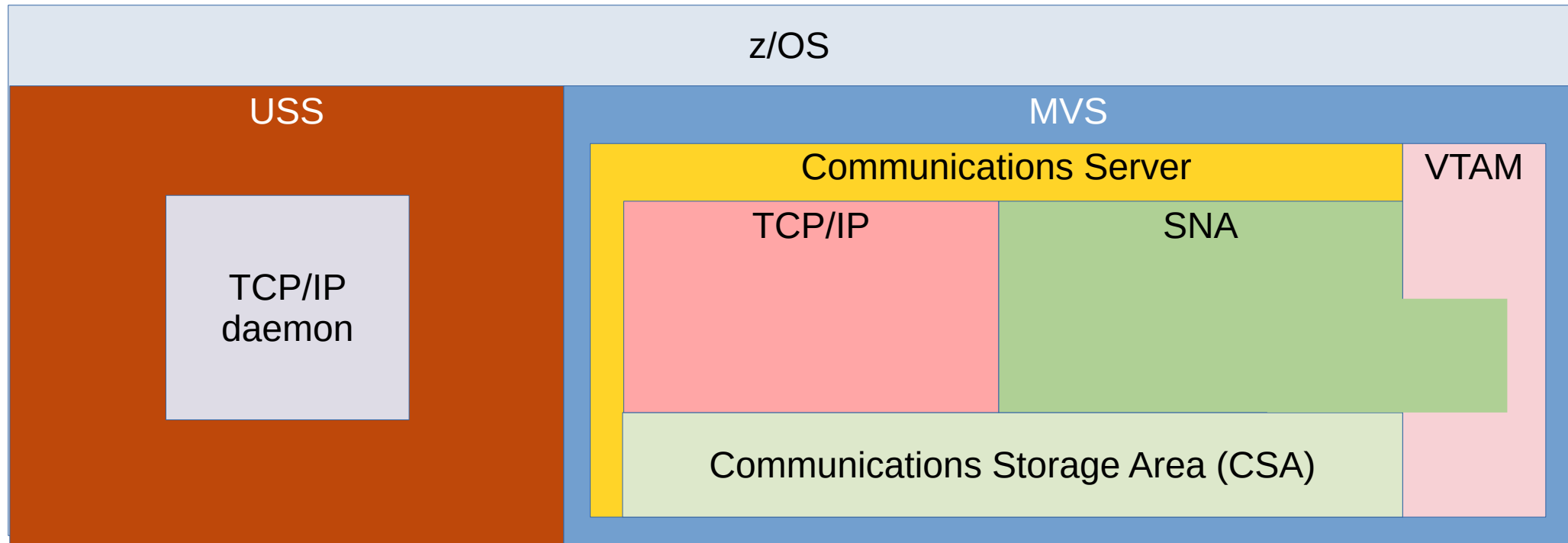
Gradually, over several releases, IBM integrated TCP/IP more and more closely with z/OS and de-emphasized SNA. Today, SNA is only used for limited application-to-application communication within a z/OS system, and in some cases to support legacy applications. TCP/IP is now the native networking stack for z/OS.

NETWORK SUPPORT ON Z/OS



As with many other improvements in z/OS, networking support is a blend of old and new technologies.

NETWORK SUPPORT ON Z/OS



The TCP/IP subsystem is initiated as a started task using JCL, the same as other z/OS subsystems. It forks a TCP/IP daemon on "the USS side of the house," which tells its parent process to kill itself after it completes its initialization. Most TCP/IP and UDP services are handled under USS, and some under MVS.

When you connect to z/OS using ssh, or when you use an off-platform tool like Ansible, you are entering the z/OS house through the USS door.

STARTING TCP/IP

A lot of configuration is possible, but in principle starting a bare-bones TCP/IP subsystem with a default configuration can be as simple as this:

```
//TCPIP    PROC
//TCPIP    EXEC  PGM=EZBTCPIP,PARM=&PARMS,REGION=0,TIME=1440
//PROFILE DD  DISP=SHR,DSN=SYS1.PARMLIB(PROFILE)
//SYSTCPD DD  DISP=SHR,DSN=SYS1.PARMLIB(TCPDATA)
```

EZBTCPIP is IBM's program for starting TCP/IP on z/OS. It reads a couple of configuration files whose DD statements you see here. That's where you would define your IP addresses and so forth. A few more DD statements are actually required, for various error output files.

```
//TCPIP    PROC
//TCPIP    EXEC  PGM=EZBTCPIP,PARM=&PARMS,REGION=0,TIME=1440
//SYSPRINT DD  SYSOUT=*
//SYSERR   DD  SYSOUT=*
//SYSERROR DD  SYSOUT=*
//ERRORFIL DD  SYSOUT=*
//SYSDEBUG DD  SYSOUT=*
//PROFILE DD  DISP=SHR,DSN=SYS1.PARMLIB(PROFILE)
//SYSTCPD DD  DISP=SHR,DSN=SYS1.PARMLIB(TCPDATA)
```

TCP/IP PROFILE

TCPIP.TCPPARMS(PROFILE) contains the TCP/IP "profile" settings. IBM supplies a sample member with default values and ample comments explaining what the settings mean.

You can view this member on the lab system, so we won't paste the whole thing on a slide. Here's the section where you define port numbers.

```
;
;   The port values below are from RFC 1060, "Assigned Numbers"
;
; commented out so any proc name can bind
; 111 TCP PMAPOE           ; Portmap Server
; 111 UDP PMAPOE           ; Portmap Server
; 2049 UDP MVS NFS         ; NFS Server

PORT
  20 TCP OMVS              NOAUTOLOG ; FTP Server
  21 TCP OMVS              ; FTP Server
  22 TCP OMVS              ; Secure Shell (SSH)
; 23 TCP TN3270            ; TELNET Server
  25 TCP SMTP              ; SMTP Server
  53 TCP NAMESRV           ; Domain Name Server
  53 UDP NAMESRV           ; Domain Name Server
 111 TCP RPCBIND1          ; Portmap Server
 111 UDP RPCBIND1          ; Portmap Server
 135 UDP NCSLLBD           ; NCS Location Broker
 161 UDP SNMP              ; SNMP Agent
 162 UDP SNMPQ             ; SNMPQ Agent
 512 TCP OMVS              ; REXEC BOSKO
 514 TCP OMVS              ; RSH BOSKO
 515 TCP LPDSRV            ; LPD Server
 520 UDP ROUTED            ; Routed Server
 750 TCP MVS KERB          ; Kerberos
 750 UDP MVS KERB          ; Kerberos
 751 TCP ADM@SRV           ; Kerberos Admin Server
 751 UDP ADM@SRV           ; Kerberos Admin Server
 992 TCP TN3270            ; Secure TELNET Server
3000 TCP CICS TCP          ; CICS Socket
5989 TCP CFZCIM            ; CIM Server
```

TCP/IP PROFILE

```
;
; Hardware definitions:
;
;DEVICE  OSAFE      MPCIPA  NONROUTER
;LINK    OSA1500    IPAQNET  OSAFE VLANID 505
INTERFACE OSA1500
  DEFINE IPAQNET
  IPADDR &HOMEIPADDRESS1
  PORTNAME &OSAPORTNAME
  INBPERF DYNAMIC
; VLANID 505
PRIROUTER
```

This section of TCPIP.TCPPARMS(PROFILE) contains hardware interface definitions.

Our goal here is not to explain everything in detail, but only to give you an idea of the kinds of information this member contains.

TCP/IP TCPDATA

The other main configuration file for TCP/IP is member TCPDATA in library TCPIP.TCPPARMS.

IBM provides a sample member that you can customize for your environment. It contains plenty of explanatory comments to help you understand the purpose of each entry.

```
; TCPIPJOBNAME specifies the name of the started procedure which was
; used to start the TCPIP address space.   TCPIP is the default.
;
TCPIPJOBNAME TCPIP
;
; HOSTNAME specifies the TCP host name of this system.  If not
; specified, the default HOSTNAME will be the node name specified
; in the IEFSSNxx PARMLIB member.
;
; For example, if this TCPIP.DATA data set is shared between two
; systems, OURMVSNAME and YOURMVSNAME, then the following
; two lines will define the HOSTNAME correctly on each system.
;
;HostName &SYSNAME.
HostName &TCPHOSTNAME_
;
; DOMAINORIGIN specifies the domain origin that will be appended
; to host names passed to the resolver.  If a host name contains
; any dots, then the DOMAINORIGIN will not be appended to the
; host name.
;
DOMAINORIGIN &TCPDOMAIN_
```

TCP/IP AND RACF

It's unlikely you would run TCP/IP out of the box. You'd want integration with z/OS security facilities, runtime monitoring, audit trail, etc. The startup job in our lab environment begins with:

```
//TCPIP    PROC PARMS='CTRACE(CTIEZB00)'  
//S1      EXEC PGM=IKJEFT01,DYNAMNBR=75,TIME=1440,  
//      PARM='RACFCRT VPCVSIIPADDRESS VSICA CERTEXPIRE'  
//SYSPRINT DD SYSOUT=*  
//SYSTSPRT DD SYSOUT=*  
//SYSTEM  DD DUMMY  
//SYSEXEC DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR  
//SYSTSIN DD *
```

```
BROWSE    SYS1.PARMLIB(CTIEZB00)                               Line 0000000000 Col 001 080  
Command ==> |                                                    Scroll ==> CSR  
***** Top of Data *****  
TRACEOPTS  
      ON  
      BUFSIZE(8M)  
***** Bottom of Data *****
```

This is turning on component tracing using values from a PARMLIB member. You'll recognize the program being executed in this step as the batch TSO program.

INTEGRATING OFF-PLATFORM APPS

When allowing external applications access to a z/OS system, we need to be careful about security. Here we see some additions to the TCP/IP startup job to check for a JWT token and optionally create a local token store for it.

We use batch TSO (IKJEFT01) to run a TSO command and batch USS (BPXBATCH) to run USS shell scripts. These are related to Ansible.

The steps must complete successfully before we attempt to run EZBTCPIP.

```
//TCPIP    PROC PARM='CTRACE(CTIEZB00)'  
//S1       EXEC PGM=IKJEFT01,DYNAMNBR=75,TIME=1440,  
//    PARM='RACFCRT VPCVSIIPADDRESS VSICA CERTEXPIRE'  
//SYSPRINT DD SYSOUT=*  
//SYSTSPRT DD SYSOUT=*  
//SYSTEM   DD DUMMY  
//SYSEXEC  DD DSN=SYS1.VS01.TS0.CLIST,DISP=SHR  
//SYSTSIN  DD *  
/* Begin Ansible Block Insert */  
//FEKJCL   EXEC PGM=IKJEFT01,DYNAMNBR=75,REGION=4M,  
//    PARM='CXXPTRL'  
//SYSPRINT DD SYSOUT=*  
//SYSTSPRT DD SYSOUT=*  
//SYSTEM   DD DUMMY  
//SYSEXEC  DD DSN=SYS1.VS01.TS0.CLIST,DISP=SHR  
//SYSTSIN  DD *  
//FEKGSK   EXEC PGM=BPXBATCH,TIME=1440,COND=(8,GT,FEKJCL),  
//    PARM='sh /usr/lpp/IBM/zexpl/bin/fek.csfpqsk.rex -NAME=JWTTOK.FEKAPPL  
//          -SIZE=256 -TYPE=CLEAR'  
//STDOUT   DD SYSOUT=*  
//*  
//IDZKEYS  EXEC PGM=BPXBATCH,TIME=1440,  
//    PARM='sh /etc/idzkeys.sh'  
//STDOUT   DD SYSOUT=*  
/* End Ansible Block Insert */  
//TCPIP    EXEC PGM=EZBTCPIP,PARM='&PARMS',REGION=0M,TIME=1440  
//SYSPRINT DD SYSOUT=*  
//SYSERR   DD SYSOUT=*  
//SYSERROR DD SYSOUT=*  
//ERRORFIL DD SYSOUT=*  
//SYSDEBUG DD SYSOUT=*  
//PROFILE  DD DISP=SHR,DSN=TCPIP.TCPPARMS(PROFILE)  
//SYSTCPD  DD DISP=SHR,DSN=TCPIP.TCPPARMS(TCPDATA)
```

INTEGRATING OFF-PLATFORM APPS

```
- name: Submit a TSO command to create a QSAM data set
  hosts: zos
  gather_facts: false
  collections:
    - ibm.ibm_zos_core
  environment: "{{ environment_vars }}"

  tasks:
    - name: Create a data set using TSO ALLOC
      ibm.ibm_zos_core.zos_tso_command:
        commands:
          - ALLOC DA(QSAM2) NEW CATALOG SPACE(1 1) TRACKS DSORG(PS)
        register: result

    - name: Display result of data set creation
      debug:
        msg: "{{ result }}"
```

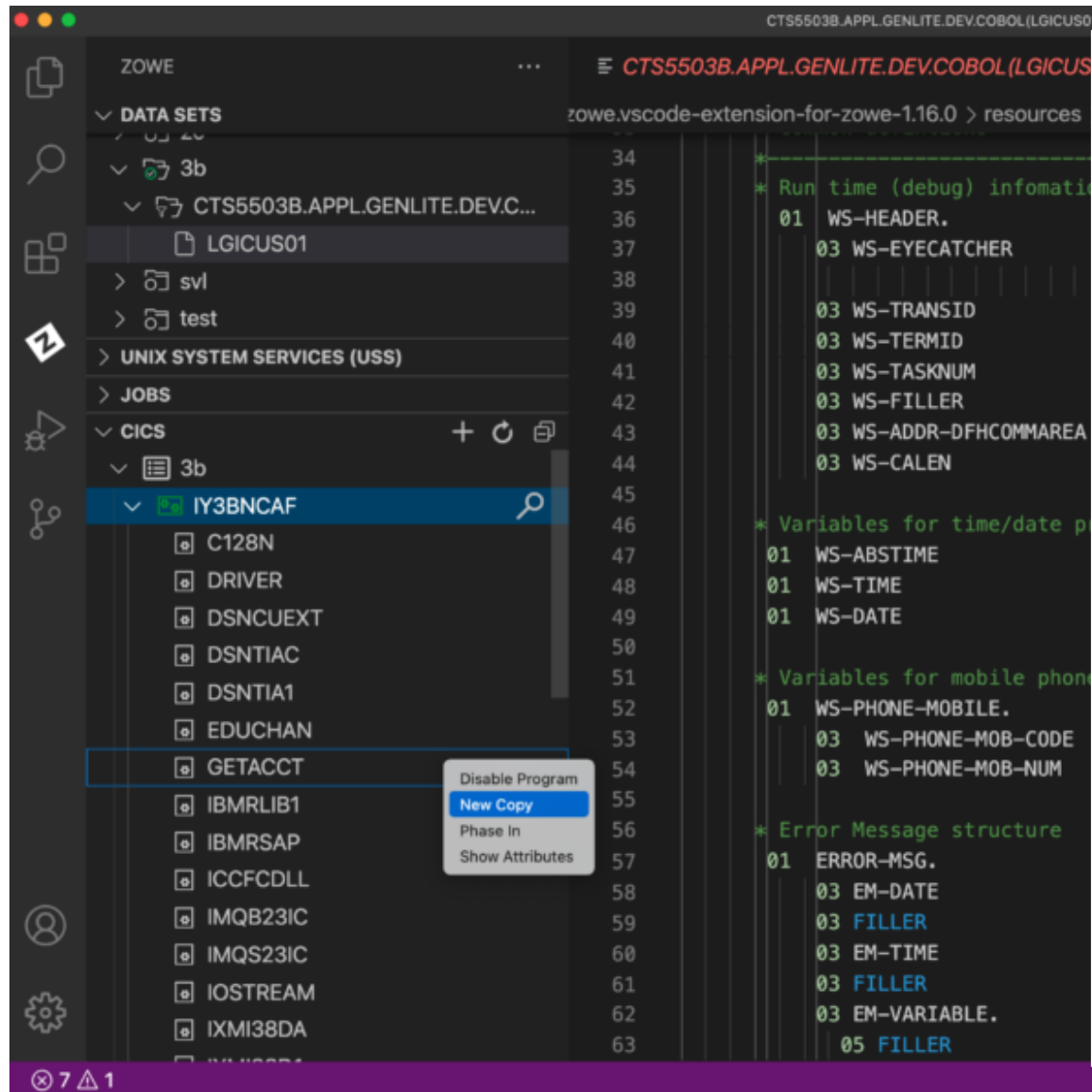
Part of an Ansible playbook that "command-line-scrapes" a TSO ALLOCATE command to create a QSAM data set on a z/OS system.

z/OS systems have to interoperate with other systems on the network much more intimately than when the original System/360 was introduced. A basic and somewhat crude way for external applications to access z/OS services is by connecting via ssh and issuing commands via the USS (omvs) command line interface.

You can connect to z/OS "manually" using ssh and then type in USS commands. Similarly, an off-platform application can connect using ssh and simulate a user typing in commands. The application then has to interpret the console output from the command.

I call it "command line scraping," because it's more-or-less as sophisticated and stable as "screen scraping."

INTEGRATING OFF-PLATFORM APPS



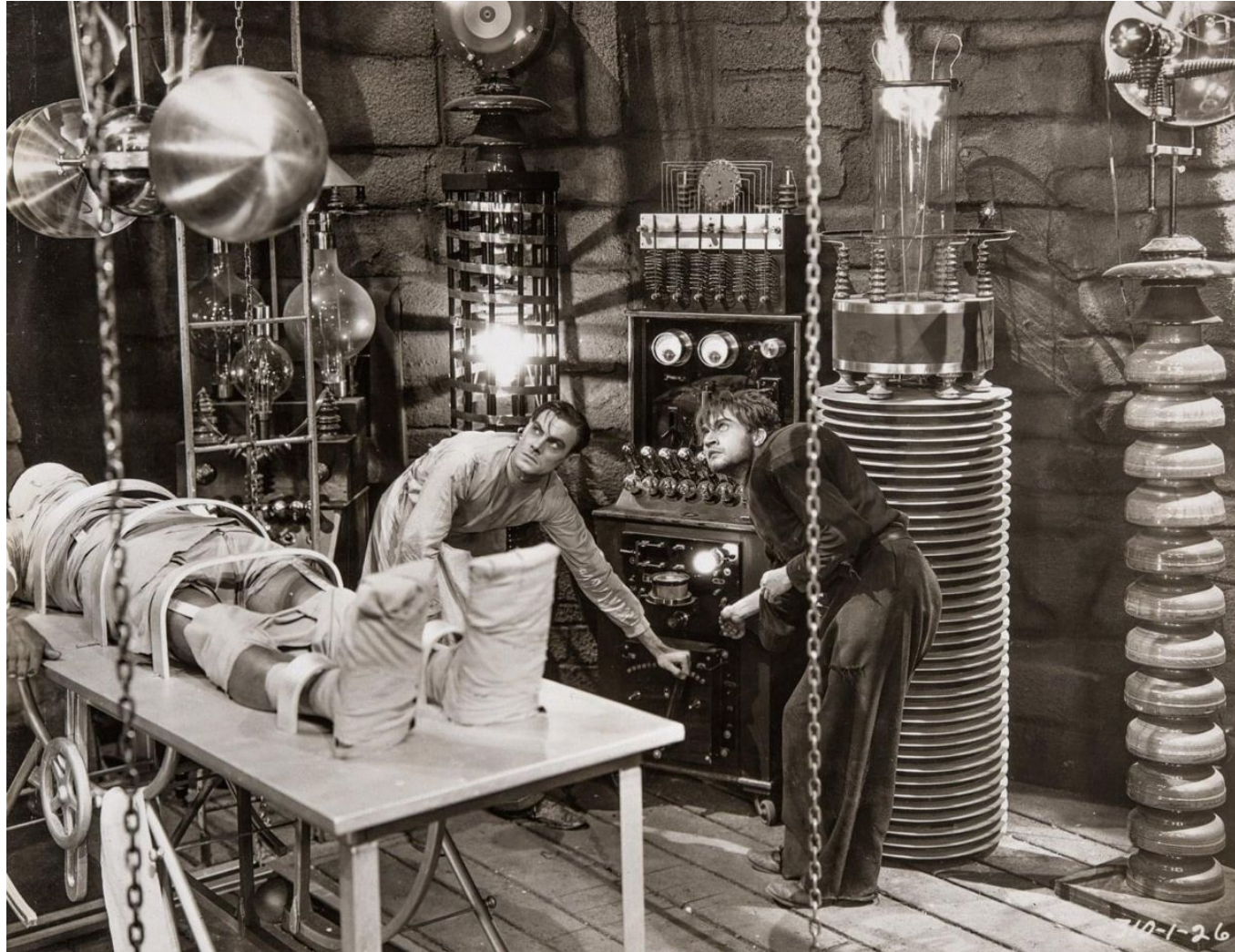
IBM has been developing APIs for z/OS services for many years now. More and more external applications are operating in a well-integrated way with z/OS systems by taking advantage of these APIs.

Probably the most prominent example as of the mid-2020s is Zowe, from Broadcom.

Zowe integrates Microsoft's VSCode with z/OS, along with VSCode extensions from IBM that provide syntax highlighting and autocompletion for JCL, Assembler, COBOL, and PL/I.

This is a screenshot of Zowe from a presentation by the Open Mainframe Project. The developer is performing a CICS "new copy" command, which would normally involve signing on to the CICS instance and using the CEMT command. With Zowe, it's a context menu option.

LAB Z/OS 34: ENABLE SSH ACCESS FOR YOURSELF



TRANSFERRING FILES



There are several common methods of transferring files between systems, and numerous ways to configure them and establish security constraints around their use.

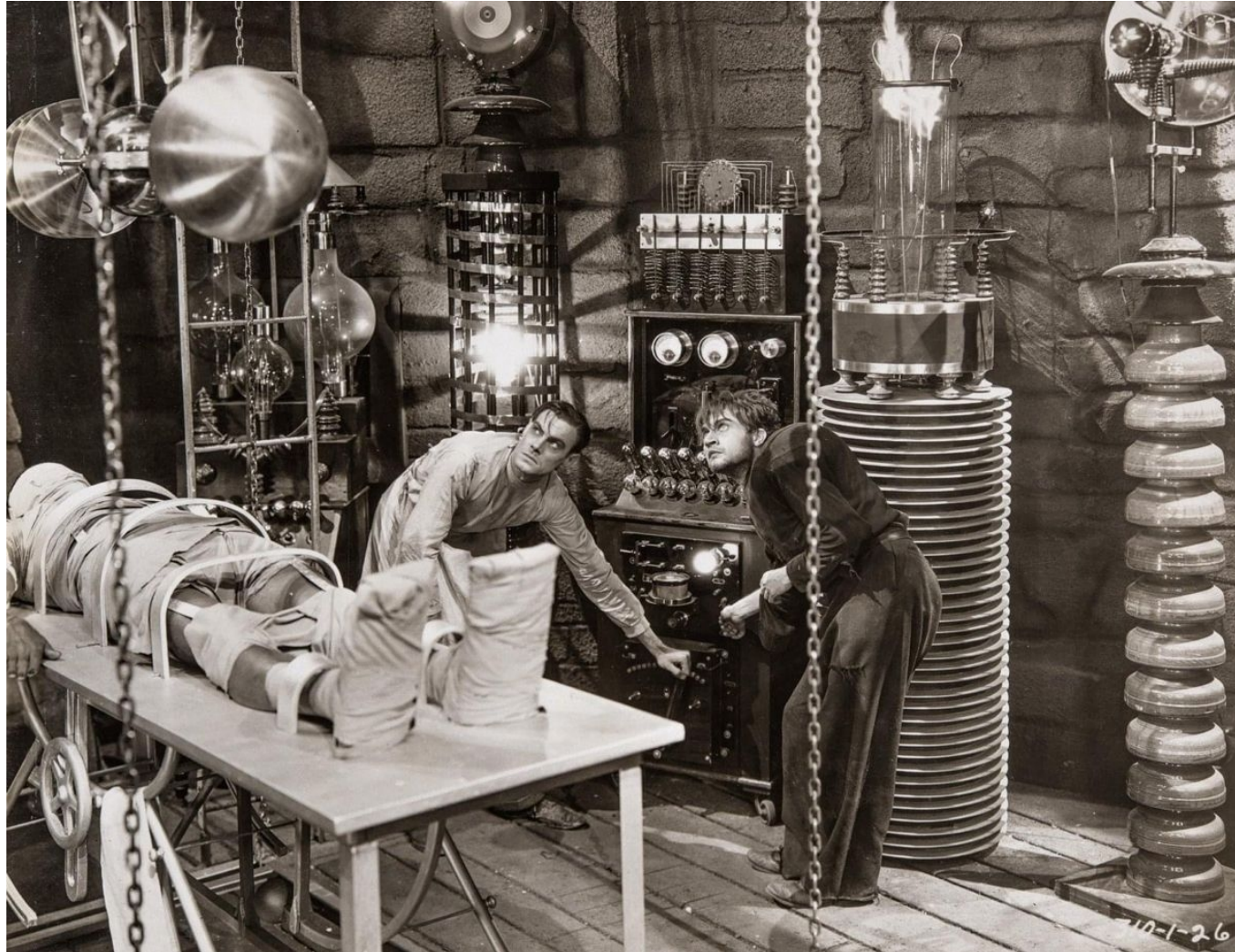
We will not be going into any of that in detail here. Whatever we might do in the lab environment is almost certainly different from what you will find on the job.

During this bootcamp, you will use (or have already used) three methods of transferring files:

- 3270 Terminal Emulators have features to invoke the TSO IND\$FILE command.
- The *nix **scp** command carries out a file transfer operation under the auspices of ssh security (same key pair you use for ssh).
- Ansible playbooks use the `zos_copy` module from the `zos_core` collection.

Modern front-ends like Zowe make it easy to transfer files. Other methods include secure FTP and IBM's Network Data Mover (NDM), as well as various third-party products.

LAB Z/OS 35: TRANSFER FILES OVER SSH WITH SCP



WHAT YOU KNOW AND WHAT YOU CAN DO

- You have a general understanding of how TCP/IP is supported on z/OS.
- Given a working TCP/IP configuration on the system, you can set up an `authorized_key` file for yourself and enable ssh access.
- You can use `scp` to transfer files securely between your local system and USS.

WHAT'S NEXT

- UNIX System Services